

Laboratorio 1: Optimización de la función de Rosenbrock

Adrián Janus Gonzalez Adamuz

26 de enero de 2026

1. La función de Rosenbrock

La función de Rosenbrock:

$$f : \mathbb{R}^2 \rightarrow \mathbb{R}$$

definida por

$$f(x, y) = (a - x)^2 + b(y - x^2)^2, \quad a > 0, b > 0. \quad (1)$$

Dicha función es no negativa para todo $(x, y) \in \mathbb{R}^2$, pues se expresa como suma de cuadrados. La geometría de su gráfica está dominada por un valle curvo que se aproxima a la parábola $y = x^2$.

1.1. Puntos estacionarios

Para determinar los puntos estacionarios de la función (1), se calcula su gradiente. Las derivadas parciales vienen dadas por

$$\frac{\partial f}{\partial x}(x, y) = -2(a - x) - 4bx(y - x^2), \quad \frac{\partial f}{\partial y}(x, y) = 2b(y - x^2).$$

Un punto (x, y) es estacionario si satisface $\nabla f(x, y) = 0$. De la ecuación $\partial f / \partial y = 0$ se obtiene

$$y = x^2.$$

Al sustituir en $\partial f / \partial x = 0$, se obtiene

$$-2(a - x) = 0,$$

lo cual implica $x = a$. En consecuencia, el único punto estacionario de la función de Rosenbrock es

$$(x^*, y^*) = (a, a^2).$$

1.2. Clasificación del punto estacionario

Para clasificar la naturaleza del punto estacionario encontrado, se considera la matriz Hessiana de f , cuyas entradas están dadas por

$$\frac{\partial^2 f}{\partial x^2} = 2 - 4by + 12bx^2, \quad \frac{\partial^2 f}{\partial x \partial y} = -4bx, \quad \frac{\partial^2 f}{\partial y^2} = 2b.$$

Evaluando la Hessiana en el punto (a, a^2) se obtiene

$$Hf(a, a^2) = \begin{pmatrix} 2 + 8ba^2 & -4ba \\ -4ba & 2b \end{pmatrix}.$$

El determinante de esta matriz es

$$\det Hf(a, a^2) = (2 + 8ba^2)(2b) - (4ba)^2 = 4b > 0,$$

y además se cumple que

$$\frac{\partial^2 f}{\partial x^2}(a, a^2) = 2 + 8ba^2 > 0.$$

El Hessiano es positivo definido, por lo que el punto (a, a^2) es un mínimo local estricto. Dado que $f(x, y) \geq 0$ para todo (x, y) y que $f(a, a^2) = 0$, dicho mínimo es además un mínimo global.

2. Problema de optimización con restricción

Se considera ahora el mismo problema de optimización, pero sujeto a la restricción de igualdad

$$x^2 + y^2 = c^2, \quad c > 0, \quad (2)$$

la cual describe una circunferencia de radio c centrada en el origen.

2.1. Lagrangiano

Para abordar el problema de optimización restringida se introduce el Lagrangiano

$$\mathcal{L}(x, y, \lambda) = (a - x)^2 + b(y - x^2)^2 + \lambda(x^2 + y^2 - c^2),$$

Las condiciones necesarias de primer orden vienen dadas por el sistema

$$\frac{\partial \mathcal{L}}{\partial x} = -2(a - x) - 4bx(y - x^2) + 2\lambda x = 0,$$

$$\frac{\partial \mathcal{L}}{\partial y} = 2b(y - x^2) + 2\lambda y = 0,$$

$$\frac{\partial \mathcal{L}}{\partial \lambda} = x^2 + y^2 - c^2 = 0.$$

Este sistema no lineal de tres ecuaciones en las incógnitas (x, y, λ) no admite, en general, una solución analítica cerrada, por lo que su resolución se lleva a cabo mediante métodos numéricos.

2.2. Resolución numérica

Al variar el parámetro c , se obtiene una familia de puntos estacionarios que describe una trayectoria en el plano (x, y) , la cual refleja cómo se desplaza el óptimo restringido conforme cambia la geometría de la restricción.

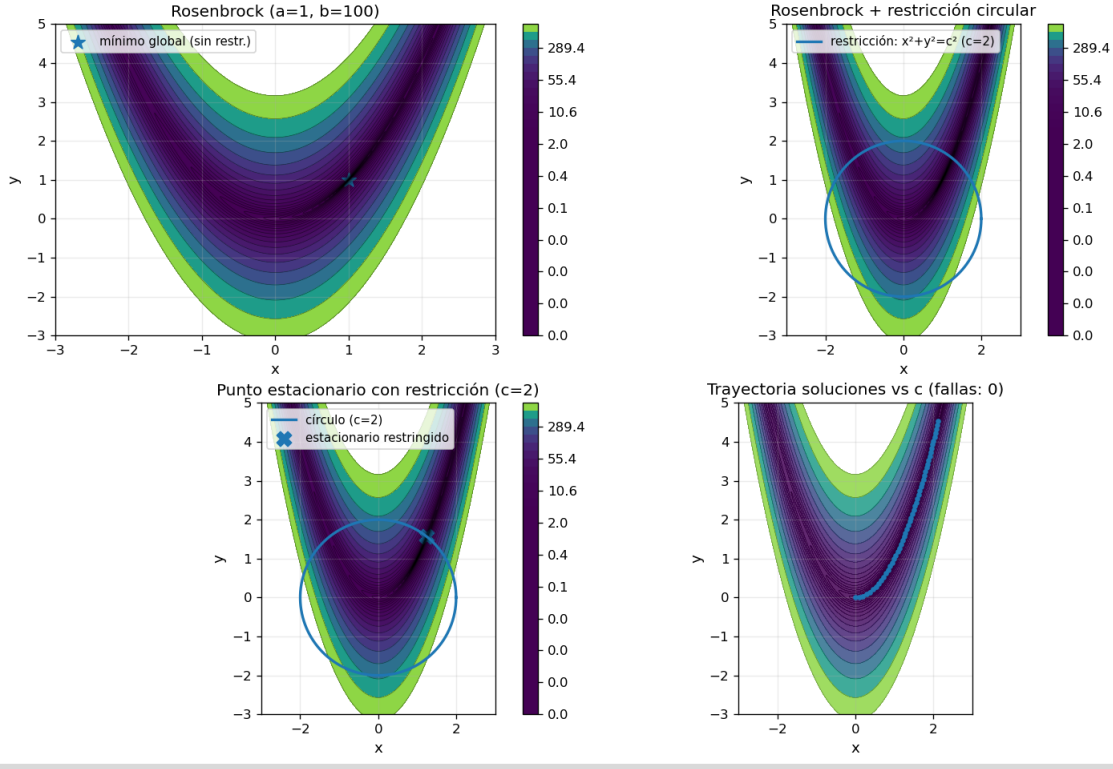


Figura 1: Resultados Completos

3. Hiperbanana

La función de generalizada de Rosenbrock:

$$f : \mathbb{R}^n \rightarrow \mathbb{R}^n$$

definida

$$f(x) = \sum_{i=1}^{N-1} ((a - x_i)^2 + b(x_{i+1} - x_i^2)^2), a > 0, b > 0$$

para $a = 1$ y $b = 100$

3.1. Caso N=3

Para $N = 3$, con $a = 1$ y $b = 100$, la función toma la forma

$$f(x_1, x_2, x_3) = (1 - x_1)^2 + 100(x_2 - x_1^2)^2 + (1 - x_2)^2 + 100(x_3 - x_2^2)^2. \quad (3)$$

Dado que $f : \mathbb{R}^3 \rightarrow \mathbb{R}$, la visualización directa mediante una superficie no es posible. En su lugar, se estudia su comportamiento geométrico mediante cortes y proyecciones, fijando una variable y analizando curvas de nivel en los planos restantes, así como evaluando la función sobre trayectorias relevantes.

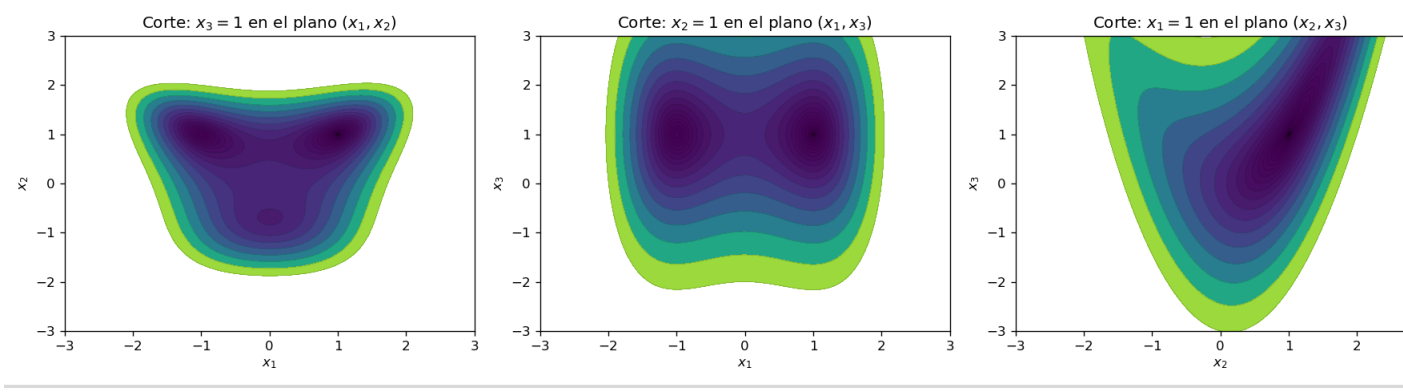


Figura 2: Resultados Completos

3.2. Condiciones de primer orden para N general

Los puntos estacionarios de f satisfacen el sistema no lineal $\nabla f(x) = 0$. Las derivadas parciales están dadas por:

$$\frac{\partial f}{\partial x_1} = -2(a - x_1) - 4b x_1(x_2 - x_1^2), \quad (4)$$

$$\frac{\partial f}{\partial x_i} = -2(a - x_i) - 4b x_i(x_{i+1} - x_i^2) + 2b(x_i - x_{i-1}^2), \quad 2 \leq i \leq N-1, \quad (5)$$

$$\frac{\partial f}{\partial x_N} = 2b(x_N - x_{N-1}^2). \quad (6)$$

Por lo tanto, el sistema de ecuaciones para un punto estacionario se escribe como

$$\nabla f(x) = 0. \quad (7)$$

Una posible solución es:

$$x_1 = a, \quad x_{i+1} = x_i^2, \quad i = 1, \dots, N-1.$$

En el caso particular $a = 1$, esta relación implica

$$x^* = (1, 1, \dots, 1),$$

el cual corresponde al mínimo global de la función, con valor $f(x^*) = 0$.

3.3. Resolución numérica del sistema sin restricción

Para $a = 1$, $b = 100$ y $N = 100$, el sistema $\nabla f(x) = 0$ es un sistema no lineal de gran dimensión que no admite solución analítica explícita. Para resolverlo numéricamente se utilizó la función `root` del módulo `scipy.optimize`, la cual implementa distintos métodos para resolver sistemas no lineales.

En particular, el problema se formula como la búsqueda de una raíz de la función

$$F(x) = \nabla f(x),$$

y se resuelve mediante un método tipo Newton o cuasi-Newton, según la configuración elegida internamente por `root`. El criterio de paro se basa en la norma del residuo $\|F(x)\|$ y en la estabilidad de las iteraciones sucesivas.

Para los puntos dados conseguimos el siguiente resultado:

Converge: False

Norma del residuo $\|grad\|$: 2.773162768811204

Primeras componentes: [0.99999675 0.99999487 0.99999281 0.99999073 0.99998886]

4. Problema con restricción esférica

Se considera ahora el mismo problema de optimización sujeto a la restricción de igualdad

$$\|x\|^2 = c^2. \quad (8)$$

Geoméricamente, esta restricción define una esfera de radio c centrada en el origen en $\mathbb{R}^N$.

4.1. Lagrangiano y condiciones de primer orden

El Lagrangiano asociado al problema es

$$\mathcal{L}(x, \lambda) = f(x) + \lambda \left(\sum_{i=1}^N x_i^2 - c^2 \right). \quad (9)$$

$$\nabla_x \mathcal{L}(x, \lambda) = \nabla f(x) + 2\lambda x = 0, \quad (10)$$

$$\sum_{i=1}^N x_i^2 = c^2. \quad (11)$$

Al escribir las ecuaciones componente a componente se obtiene un sistema no lineal de $N + 1$ ecuaciones. Dependiendo del valor del multiplicador de Lagrange λ y de la posible presencia de componentes nulas en x , el sistema presenta tres casos cualitativamente distintos, los cuales deben tratarse con cuidado en el análisis numérico.

4.2. Resolución numérica del sistema KKT

El sistema de condiciones de primer orden se resolvió numéricamente utilizando nuevamente la función `root` de `scipy.optimize`, aplicada ahora al sistema aumentado

$$F(x, \lambda) = \begin{pmatrix} \nabla f(x) + 2\lambda x \\ \sum_{i=1}^N x_i^2 - c^2 \end{pmatrix}.$$

Este enfoque corresponde a aplicar un método de Newton al sistema KKT del problema restringido. La convergencia depende de manera sensible de la elección del punto inicial, tanto para x como para λ , así como del valor del parámetro c .

```

Converge: False
||grad f + 2*lambda*x||: 1.3006886874590542
||x||^2: 4.085219067280299    c^2: 4.0

```

4.3. Barrido en el parámetro c

El procedimiento anterior se repitió para distintos valores del parámetro c , con $c \in \{0, 0.1, 0.2, \dots, 5\}$. Para cada valor de c se obtuvo un punto estacionario factible $x^*(c)$.

Dado que la restricción impone $\|x^*(c)\| = c$, la distancia del mínimo al origen coincide necesariamente con el valor de c . Por esta razón, además de verificar esta relación numéricamente, resulta más informativo analizar el comportamiento del valor óptimo $f(x^*(c))$ o la distancia $\|x^*(c) - \mathbf{1}\|$, donde $\mathbf{1} = (1, \dots, 1)$ es el minimizador no restringido. Estos resultados permiten cuantificar cómo la restricción geométrica afecta la localización y el valor del mínimo.

Total fallas: 9

Último c exitoso: 4.4

Resultados por valor de c :

```

c = 0.0 -> Éxito
c = 0.1 -> Éxito
c = 0.2 -> Éxito
c = 0.3 -> Éxito
c = 0.4 -> Éxito
c = 0.5 -> Éxito
c = 0.6 -> Falla
c = 0.7 -> Falla
c = 0.8 -> Falla
c = 0.9 -> Éxito
c = 1.0 -> Éxito
c = 1.1 -> Éxito
c = 1.2 -> Éxito
c = 1.3 -> Éxito
c = 1.4 -> Éxito
c = 1.5 -> Éxito
c = 1.6 -> Éxito
c = 1.7 -> Éxito
c = 1.8 -> Éxito
c = 1.9 -> Éxito
c = 2.0 -> Éxito
c = 2.1 -> Éxito
c = 2.2 -> Éxito
c = 2.3 -> Éxito

```

```
c = 2.4 -> Éxito
c = 2.5 -> Éxito
c = 2.6 -> Éxito
c = 2.7 -> Éxito
c = 2.8 -> Éxito
c = 2.9 -> Éxito
c = 3.0 -> Éxito
c = 3.1 -> Éxito
c = 3.2 -> Éxito
c = 3.3 -> Éxito
c = 3.4 -> Éxito
c = 3.5 -> Éxito
c = 3.6 -> Éxito
c = 3.7 -> Éxito
c = 3.8 -> Éxito
c = 3.9 -> Éxito
c = 4.0 -> Éxito
c = 4.1 -> Éxito
c = 4.2 -> Éxito
c = 4.3 -> Éxito
c = 4.4 -> Éxito
c = 4.5 -> Falla
c = 4.6 -> Falla
c = 4.7 -> Falla
c = 4.8 -> Falla
c = 4.9 -> Falla
c = 5.0 -> Falla
```

A. Código para visualización y problema no restringido

```
1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import fsolve
4
5 plt.rcParams.update({
6     "figure.figsize": (12, 9),
7     "figure.dpi": 120,
8     "axes.grid": True,
9     "grid.alpha": 0.25,
10    "axes.titlesize": 12,
11    "axes.labelsize": 11,
12    "legend.fontsize": 9,
13    "font.size": 10,
14    "lines.linewidth": 2.0
```

```

15 })
16
17 def rosenbrock(x, y, a=1.0, b=100.0):
18     return (a-x)**2 + b*(y-x**2)**2
19
20 def Lag(vars_, a, b, c):
21     x, y, lam = vars_
22     eq1 = -2*(a-x) - 4*b*x*(y-x**2) + 2*lam*x
23     eq2 = 2*b*(y-x**2) + 2*lam*y
24     eq3 = x**2 + y**2 - c**2
25     return [eq1, eq2, eq3]
26
27 # =====
28 # Parametros
29 a, b = 1.0, 100.0
30 c = 2.0
31
32 # logspace
33 niveles = np.logspace(-4, 3, 40)
34
35 # Malla 1 (banana clasica)
36 xmin, xmax = -3.0, 3.0
37 ymin, ymax = -3.0, 5.0
38 X= np.linspace(xmin, xmax, 700)
39 Y = np.linspace(ymin, ymax, 700)
40 XX, YY = np.meshgrid(X, Y)
41 ZZ = rosenbrock(XX, YY, a=a, b=b)
42
43
44
45 # Punto estacionario sin restriccion
46 x_sta, y_sta = a, a**2
47
48 # Solucion numerica restringida (para c=2)
49 x0 = np.array([1.0, 1.0, 0.0])
50 sol = fsolve(Lag, x0, args=(a, b, c))
51 x_sol, y_sol, lam_sol = sol
52
53 # Parametrizacion del circulo
54 t = np.linspace(0, 2*np.pi, 800)
55 xc = c*np.cos(t)
56 yc = c*np.sin(t)
57
58 # =====
59 # Subplots: 2x2
60 fig, axs = plt.subplots(2, 2)
61
62 # (1) Banana + minimo sin restriccion

```

```

63 ax = axs[0, 0]
64 cf = ax.contourf(XX, YY, ZZ, levels=niveles, cmap="viridis")
65 ax.contour(XX, YY, ZZ, levels=niveles, colors="k", linewidths=0.35,
        alpha=0.6)
66 ax.scatter([x_sta], [y_sta], s=120, marker="*", label="minimo global (
        sin restr.)")
67 ax.set_title("Rosenbrock (a=1, b=100)")
68 ax.set_xlabel("x")
69 ax.set_ylabel("y")
70 ax.set_xlim(xmin, xmax)
71 ax.set_ylim(ymin, ymax)
72 ax.legend(loc="upper left")
73 fig.colorbar(cf, ax=ax)
74
75 # (2) Banana + circulo (c=2)
76 ax = axs[0, 1]
77 cf = ax.contourf(XX, YY, ZZ, levels=niveles, cmap="viridis")
78 ax.contour(XX, YY, ZZ, levels=niveles, colors="k", linewidths=0.4, alpha
        =0.5)
79 ax.plot(xc, yc, label="restriccion:  $x^2+y^2=c^2$  (c=2)")
80 ax.set_title("Rosenbrock + restriccion circular")
81 ax.set_xlabel("x")
82 ax.set_ylabel("y")
83 ax.set_aspect("equal", adjustable="box")
84 ax.set_xlim(xmin, xmax)
85 ax.set_ylim(ymin, ymax)
86 ax.legend(loc="upper left")
87 fig.colorbar(cf, ax=ax)
88
89 # (3) Optimo con restriccion (c=2) marcado
90 ax = axs[1, 0]
91 cf = ax.contourf(XX, YY, ZZ, levels=niveles, cmap="viridis")
92 ax.contour(XX, YY, ZZ, levels=niveles, colors="k", linewidths=0.4, alpha
        =0.5)
93 ax.plot(xc, yc, label="circulo (c=2)")
94 ax.scatter([x_sol], [y_sol], s=120, marker="X", label="estacionario
        restringido")
95 ax.set_title("Punto estacionario con restriccion (c=2)")
96 ax.set_xlabel("x")
97 ax.set_ylabel("y")
98 ax.set_aspect("equal", adjustable="box")
99 ax.set_xlim(xmin, xmax)
100 ax.set_ylim(ymin, ymax)
101 ax.legend(loc="upper left")
102 fig.colorbar(cf, ax=ax)
103
104 # (4) Trayectoria de soluciones para varios c
105 ax = axs[1, 1]

```

```

106 cs = np.round(np.arange(0.0, 5.0 + 1e-12, 0.1), 1)
107 soluciones = []
108 exitos = []
109
110 seed = [
111     np.array([1.0, 1.0, 0.0]),
112     np.array([1.0, -1.0, 0.0]),
113     np.array([-1.0, 1.0, 0.0]),
114     np.array([-1.0, -1.0, 0.0]),
115     np.array([0.5, 0.0, 0.0]),
116     np.array([0.0, 0.5, 0.0]),
117 ]
118 prev = np.array([1.0, 1.0, 0.0])
119
120 for c_i in cs:
121     if c_i == 0.0:
122         soluciones.append([0.0, 0.0, 0.0])
123         exitos.append(True)
124         prev = np.array([0.0, 0.0, 0.0])
125         continue
126
127     found = False
128     intento = [prev] + seed
129     for x0 in intento:
130         sol_i, info, ier, msg = fsolve(Lag, x0, args=(a, b, c_i),
131                                     full_output=True, maxfev=2000)
132         if ier == 1:
133             x_i, y_i, lam_i = sol_i
134             if abs(x_i**2 + y_i**2 - c_i**2) < 1e-6:
135                 soluciones.append(sol_i)
136                 exitos.append(True)
137                 prev = sol_i
138                 found = True
139                 break
140
141     if not found:
142         soluciones.append([np.nan, np.nan, np.nan])
143         exitos.append(False)
144
145 soluciones = np.array(soluciones)
146 ax.contourf(XX, YY, ZZ, levels=niveles, cmap="viridis", alpha=0.85)
147 ax.contour(XX, YY, ZZ, levels=niveles, colors="k", linewidths=0.35,
148           alpha=0.4)
149 ax.plot(soluciones[:, 0], soluciones[:, 1], marker="o", markersize=3)
150
151 failed = np.sum(~np.array(exitos))
152 ax.set_title(f"Trayectoria soluciones vs c (fallas: {failed})")
153 ax.set_xlabel("x")

```

```

152 ax.set_ylabel("y")
153 ax.set_aspect("equal", adjustable="box")
154
155 plt.tight_layout()
156 plt.show()

```

B. Código para el problema restringido y método de continuación

```

1 import numpy as np
2 import matplotlib.pyplot as plt
3 from scipy.optimize import root
4 def Rosen_N (x, a=1.0, b=100.0):
5     x = np.asarray(x)
6     return np.sum((a-x[:-1])**2 + b*(x[1:] - x[:-1]**2)**2)
7 a,b = 1.0, 100.0
8 grid = np.linspace(-3, 3, 500)
9 niveles = np.logspace (-3, 3, 40)
10
11 fig, axs = plt.subplots(1,3, figsize = (15,4), dpi = 120)
12
13 #=====
14 #Visualizacion
15 #Corte 1: x3 = 1, plano (x1,x2)
16 X1, X2 = np.meshgrid(grid, grid)
17 Z = np.zeros_like(X1)
18 x3prov = 1.0
19 for i in range(X1.shape[0]):
20     Z[i,:] = [Rosen_N([X1[i,j], X2[i,j], x3prov], a=a, b=b) for j in
                range(X1.shape[1])]
21 axs[0].contourf(X1, X2, Z, levels=niveles, cmap="viridis")
22 axs[0].contour(X1, X2, Z, levels=niveles, colors="k", linewidths=0.25,
                alpha=0.35)
23 axs[0].set_title(r"Corte: $x_3=1$ en el plano $(x_1,x_2)$")
24 axs[0].set_xlabel(r"$x_1$")
25 axs[0].set_ylabel(r"$x_2$")
26
27 # corte 2: x2=1, plano (x1,x3)
28 X1, X3 = np.meshgrid(grid, grid)
29 Z = np.zeros_like(X1)
30 x2fix = 1.0
31 for i in range(X1.shape[0]):
32     Z[i,:] = [Rosen_N([X1[i,j], x2fix, X3[i,j]], a=a, b=b) for j in
                range(X1.shape[1])]
33 axs[1].contourf(X1, X3, Z, levels=niveles, cmap="viridis")
34 axs[1].contour(X1, X3, Z, levels=niveles, colors="k", linewidths=0.25,
                alpha=0.35)

```

```

35  axs[1].set_title(r"Corte: $x_2=1$ en el plano $(x_1,x_3)$")
36  axs[1].set_xlabel(r"$x_1$")
37  axs[1].set_ylabel(r"$x_3$")
38
39  #Corte 3: x1 = 1 plano (x2, x3)
40  X2, X3 = np.meshgrid(grid, grid)
41  Z = np.zeros_like(X2)
42  x1fix = 1.0
43  for i in range(X2.shape[0]):
44      Z[i,:] = [Rosen_N([x1fix, X2[i,j], X3[i,j]], a=a, b=b) for j in
                  range(X2.shape[1])]
45  axs[2].contourf(X2, X3, Z, levels=niveles, cmap="viridis")
46  axs[2].contour(X2, X3, Z, levels=niveles, colors="k", linewidths=0.25,
                  alpha=0.35)
47  axs[2].set_title(r"Corte: $x_1=1$ en el plano $(x_2,x_3)$")
48  axs[2].set_xlabel(r"$x_2$")
49  axs[2].set_ylabel(r"$x_3$")
50
51  plt.tight_layout()
52  plt.show()
53
54  #=====

55  #Resolucion numerica
56  def grad_rosen_N(x, a=1.0, b=100.0):
57      x = np.asarray(x)
58      N = x.size
59      g = np.zeros_like(x)
60      #caso i = 0
61      g[0] = -2*(a - x[0]) - 4*b*x[0]*(x[1] - x[0]**2)
62      #caso 1<= i <= N-2
63      for i in range (1, N-1):
64          g[i] = -2*(a-x[i]) - 4*b*x[i]*(x[i+1] - x[i]**2) + 2*b*(x[i] - x
              [i-1]**2)
65      #caso i = N-1
66      g[N-1] = 2*b*(x[N-1]-x[N-2]**2)
67      return g
68  a, b = 1.0, 100.0
69  N=100
70
71  x0 = np.ones(N) * 0.8
72
73  sol = root(lambda x: grad_rosen_N(x, a=a, b=b), x0, method = "hybr")
74  print("Converge: ", sol.success)
75  print("Norma del residuo ||grad||:", np.linalg.norm(sol.fun))
76  print("Primeras componentes: ", sol.x[:5])
77

```

```

78 #=====
79 #Con Restricciones ||x||^2 = c^2
80
81 def system_constrained(Z,a = 1.0, b =100.0, c=2.0):
82     x= Z[:-1]
83     lam = Z[-1]
84     g = grad_rosen_N(x,a=a, b=b) + 2*lam*x
85     h = np.dot(x,x) - c**2
86     return np.hstack([g,h])
87 a,b = 1.0, 100.0
88 N=100
89 c = 2.0
90
91 x0 = (c/np.sqrt(N)) * np.ones(N)
92 lam0 = 0.0
93 z0 = np.hstack([x0, lam0])
94
95 sol = root (lambda z: system_constrained(z, a=a, b=b, c=c), z0, method="
    hybr")
96 x_sol = sol.x[:-1]
97 lam_sol = sol.x[-1]
98
99 print("Converge:", sol.success)
100 print("||grad f + 2lamx||:", np.linalg.norm(grad_rosen_N(x_sol, a=a, b=b
    ) + 2*lam_sol*x_sol))
101 print("||x||^2:", np.dot(x_sol, x_sol), " c^2:", c**2)
102 #=====
103
104 #Secuencias de c
105 cs = np.round(np.arange(0.0, 5.0 + 1e-12, 0.1), 1)
106 rng = np.random.default_rng(123)
107
108 def generate_seeds(N, c, n_seeds=6):
109     seeds = []
110     for _ in range(n_seeds):
111         v = rng.normal(size=N)
112         v /= np.linalg.norm(v)
113         seeds.append(c * v)
114     return seeds
115
116 N = 100
117 a, b = 1.0, 100.0
118
119 soluciones = []
120 exitos = []
121

```

```

122
123 prev_x = np.zeros(N)
124 prev_lam = 0.0
125
126 for c_i in cs:
127
128
129     if c_i == 0.0:
130         soluciones.append(np.hstack([np.zeros(N), 0.0]))
131         exitos.append(True)
132         prev_x = np.zeros(N)
133         prev_lam = 0.0
134         continue
135
136     found = False
137     seeds = generate_seeds(N, c_i, n_seeds=6)
138     attempts = [prev_x] + seeds
139
140     for x0 in attempts:
141         z0 = np.hstack([x0, prev_lam])
142
143         sol = root(
144             system_constrained,
145             z0,
146             args=(a, b, c_i),
147             method="hybr",
148             tol=1e-10
149         )
150
151         if sol.success:
152             x_sol = sol.x[:-1]
153             lam_sol = sol.x[-1]
154
155
156             if abs(np.dot(x_sol, x_sol) - c_i**2) < 1e-6:
157                 soluciones.append(sol.x)
158                 exitos.append(True)
159                 prev_x = x_sol
160                 prev_lam = lam_sol
161                 found = True
162                 break
163
164         if not found:
165             soluciones.append(np.full(N + 1, np.nan))
166             exitos.append(False)
167 soluciones = np.array(soluciones)
168 exitos = np.array(exitos)
169

```

```
170 print("Total fallas:", np.sum(~exitos))
171 print("Ultimo c exitoso:", cs[exitos][-1] if np.any(exitos) else None)
172 print("\nResultados por valor de c:")
173 for c_i, ok in zip(cs, exitos):
174     estado = "Éxito" if ok else "Falla"
175     print(f"c = {c_i:4.1f} -> {estado}")
```